ByteMart Digital

THE COMPLETE GUIDE

Docker & Kubernetes

Containers & Orchestration for Java Developers

A complete, example-driven guide to containers and orchestration for Java developers — from Dockerfiles and multi-stage builds to Kubernetes deployments, probes, scaling, and production.

24	110+	12	6
Chapters	Code Examples	Diagrams	Modules

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Docker & Kubernetes for Java Developers

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
The information in this book is provided in good faith and for educational purposes. While every effort has been made to ensure accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from the use of the code or information contained herein. All code examples are provided "as is".

Trademarks
Docker, Kubernetes, Spring, and other product names, logos, and brands referenced are the property of their respective owners and are used for identification purposes only. This guide is an independent educational work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

1. Why Containers for Java Developers 4
2. Docker Fundamentals 7
3. Your First Dockerfile for a Java App 10
4. Building & Running Containers 13
5. Layered & Multi-Stage Builds 15
6. The JVM in Containers 18
7. Building Images without a Dockerfile 20
8. Image Optimization & Security 22
9. Docker Compose for Local Dev 24
10. Why Kubernetes 27
11. Kubernetes Architecture 29
12. Pods, ReplicaSets & Deployments 32
13. Services & Networking 35
14. ConfigMaps & Secrets 38
15. Deploying a Spring Boot App 41
16. Health Probes for Spring Boot 43
17. Resource Requests & Limits 46
18. Scaling: Manual & Autoscaling 48
19. Configuration & Profiles 51
20. Rolling Updates & Rollbacks 53
21. Ingress & Exposing Services 56
22. Observability 59
23. Helm, Kustomize & GitOps 61
24. Security & Best Practices 63

MODULE 1 · CONTAINERS & DOCKER

CHAPTER 01

Why Containers for Java Developers

The 'works on my machine' problem, and how containers make a Java app portable and reproducible.

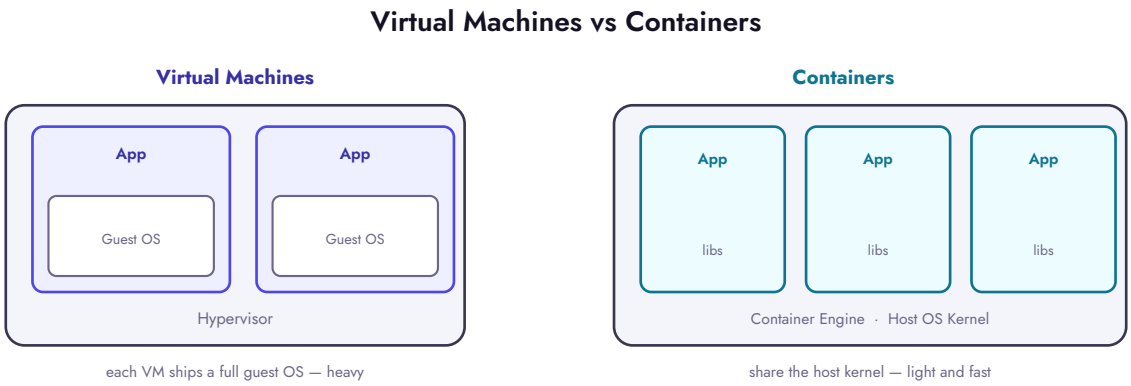
Every Java developer has heard 'but it works on my machine'. Containers end that argument by packaging your application together with everything it needs to run — the same, everywhere.

> **Container**
> A lightweight, isolated unit that bundles an application with its runtime, libraries, and dependencies, running on a shared host OS kernel — consistent across a developer laptop, CI, and production.

The JVM does not go far enough

'Write once, run anywhere' solved bytecode portability, but not the environment: the JDK version, OS libraries, environment variables, and config still vary between machines. A container captures all of that, so the artifact you test is exactly the artifact you ship.

Containers vs virtual machines



VMs ship a full guest OS each; containers share the host kernel, making them light and fast.

A VM virtualises hardware and runs a whole guest OS — heavy and slow to start. A container shares the host kernel and isolates only the process and its files — megabytes instead of gigabytes, and starting in seconds.

Why it matters for Java

- **Reproducibility** — the exact JDK and dependencies travel with the app.
- **Fast, consistent deploys** — the same image runs in every environment.
- **Microservices-ready** — each service is an independent, deployable image.
- **Foundation for Kubernetes** — orchestration needs containers to schedule.

© 2026 ByteMart Digital

Package the environment, not just the jar

Containers bundle your Java app with its exact runtime and dependencies, ending environment drift. They are lighter than VMs and the essential building block for Kubernetes.

© 2026 ByteMart Digital

MODULE 1 · CONTAINERS & DOCKER

CHAPTER 02

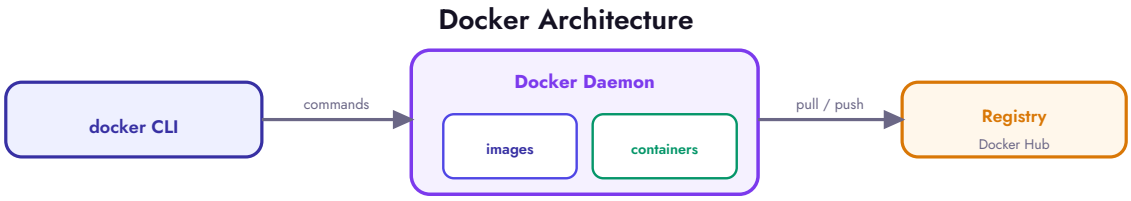
Docker Fundamentals

Images, containers, registries, and the daemon — the core concepts you will use daily.

Docker is the most common tool for building and running containers. A handful of concepts — image, container, registry, daemon — cover almost everything you will do.

The core objects

Concept	What it is
Image	a read-only template (your app + runtime)
Container	a running instance of an image
Registry	a store of images (Docker Hub, ECR)
Dockerfile	the recipe to build an image



The CLI sends commands to the daemon, which builds images and runs containers, pulling and pushing images from a registry.

The CLI sends commands to the daemon, which builds images, runs containers, and talks to a registry.

Image vs container

The relationship mirrors a class and an object: an **image** is the immutable template; a **container** is a running instance of it. One image can run as many identical containers, each isolated.

The everyday commands

bash

```
docker build -t myapp:1.0 .     # build an image from a Dockerfile
docker run -p 8080:8080 myapp:1.0  # run it, mapping a port
docker ps                       # list running containers
docker logs <id>                # view a container's logs
docker push myapp:1.0           # publish to a registry
```

Image, container, registry

An image is a template, a container is a running instance, and a registry stores images. You build with a Dockerfile and run with docker — these few concepts underpin everything.

MODULE 1 · CONTAINERS & DOCKER

CHAPTER 03

Your First Dockerfile for a Java App

Writing a Dockerfile that packages a Spring Boot jar into a runnable image.

A **Dockerfile** is a step-by-step recipe for building an image. For a Java app, the essentials are: pick a base image, add your jar, and define how to start it.

A minimal Dockerfile

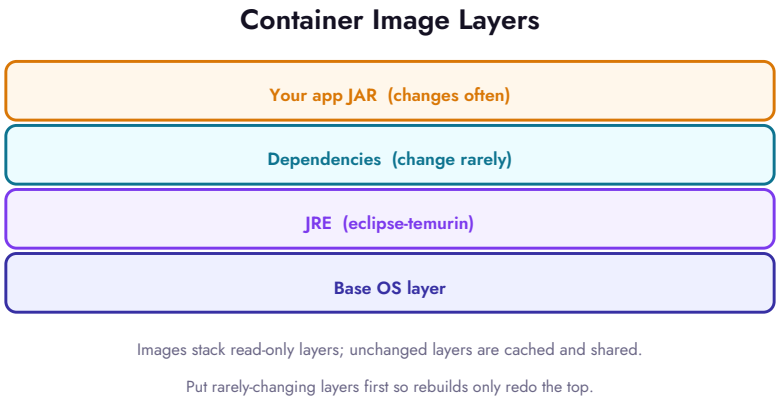
dockerfile

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/app.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

What each instruction does

- `FROM` — the base image (a JRE, not the full JDK, for runtime).
- `WORKDIR` — the working directory inside the container.
- `COPY` — add the built jar into the image.
- `EXPOSE` — document the port the app listens on.
- `ENTRYPOINT` — the command that runs when the container starts.

Images are layered



Images stack read-only layers; unchanged layers are cached, so order them stable-first.

Each instruction adds a cached layer. Docker reuses unchanged layers on rebuild, so ordering matters: put things that rarely change (the base, dependencies) before things that change often (your jar) to keep rebuilds fast.

© 2026 ByteMart Digital

TIP

Use a specific base-image tag (`21-jre`), never `latest`. Reproducible builds require pinned versions — `latest` can change under you.

A recipe in layers

A Dockerfile packages your jar onto a JRE base image with an entrypoint. Order instructions stable-first so Docker's layer cache keeps rebuilds fast, and pin base-image versions.

© 2026 ByteMart Digital

MODULE 1 · CONTAINERS & DOCKER

CHAPTER 04

Building & Running Containers

Building images, running containers, mapping ports and volumes, and passing configuration.

With a Dockerfile in place, the build-and-run cycle is quick. A few flags cover ports, environment variables, and persistent data — the things a Java app needs.

Build and run

```bash
docker build -t bytemart/orders:1.0 .
docker run -d --name orders -p 8080:8080 bytemart/orders:1.0
```

`-d` runs it in the background, `--name` gives it a friendly name, and `-p host:container` maps a host port to the container's port.

Passing configuration

Containers should be configured from the outside, not baked in. Pass environment variables that Spring Boot picks up (relaxed binding maps them to properties).

```bash
docker run -p 8080:8080 \
  -e SPRING_PROFILES_ACTIVE=prod \
  -e SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/orders \
  bytemart/orders:1.0
```

Volumes for persistent data

A container's filesystem is ephemeral — it vanishes when the container is removed. Mount a **volume** to persist data (a database's files, uploaded content) beyond the container's life.

```bash
docker run -v pgdata:/var/lib/postgresql/data postgres:16
```

WATCH OUT

Never store important data inside the container's writable layer — it is lost on restart or redeploy. Use volumes for anything that must survive.

Run with ports, env, and volumes

Build with docker build, run with docker run, map ports with -p, configure with -e environment variables, and persist data with -v volumes — keeping the image itself stateless.

MODULE 2 · CONTAINERIZING JAVA WELL

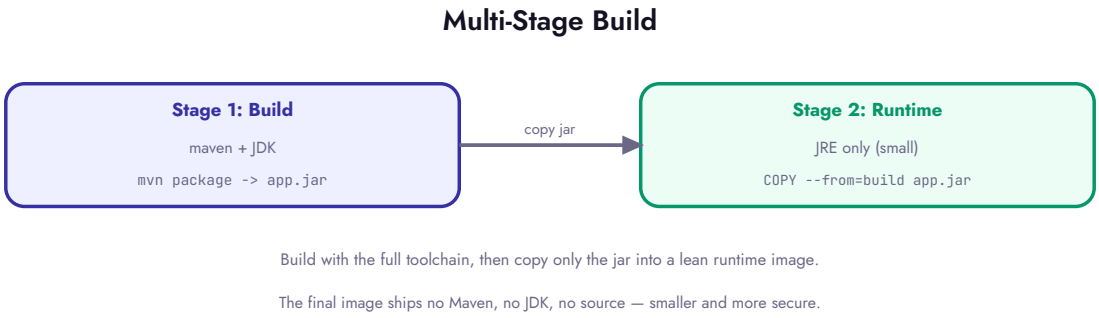
CHAPTER 05

Layered & Multi-Stage Builds

Building small, cache-friendly Java images by separating the build from the runtime.

A naive Java image is often bloated — it ships the JDK, Maven, and source alongside the app. **Multi-stage builds** fix this by building in one stage and copying only the result into a lean runtime stage.

The multi-stage pattern



Build with the full toolchain, then copy only the jar into a small runtime image.

dockerfile

```
# Stage 1: build
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /src
COPY pom.xml .
RUN mvn dependency:go-offline      # cache dependencies
COPY src ./src
RUN mvn -q clean package -DskipTests

# Stage 2: runtime
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /src/target/app.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Why it is better

- The final image has **no JDK, Maven, or source** — smaller and more secure.
- Copying `pom.xml` and resolving dependencies first caches them across rebuilds.
- Only the last stage ships, so build tools never reach production.

Spring Boot layered jars

Spring Boot can split its jar into layers (dependencies, snapshot deps, resources, application) so Docker caches the slow-changing dependency layers separately from your fast-changing code — dramatically faster

rebuilds and pushes.

> **Build big, ship small**
> Multi-stage builds compile with the full toolchain then copy only the jar into a lean JRE image. Combined with layered jars, they keep Java images small, secure, and cache-friendly.

© 2026 ByteMart Digital

MODULE 2 · CONTAINERIZING JAVA WELL

CHAPTER 06

The JVM in Containers

Making the JVM respect container memory and CPU limits so it does not get killed.

A container-unaware JVM sees the whole host's memory and CPU, not the container's limits — and sizes its heap accordingly. In a limited container that leads to the dreaded out-of-memory kill.

The historical problem

Older JVMs ignored cgroup limits and assumed all host memory was available, so a JVM in a 512MB container might try to use gigabytes of heap and be killed by the container runtime.

Container awareness

Modern JVMs (Java 10+, backported to later Java 8 updates) are **container-aware** by default — they read the container's memory and CPU limits and size themselves accordingly. This behaviour is on automatically.

Sizing the heap

Rather than a fixed `-Xmx`, prefer percentage flags so the heap scales with the container's memory limit — leaving headroom for non-heap memory (metaspace, threads, native).

bash

```
java -XX:MaxRAMPercentage=75.0 -jar app.jar
# heap = 75% of the container memory limit
```

WATCH OUT

The JVM needs more than heap: metaspace, thread stacks, code cache, and native buffers all use memory. Never set the heap to 100% of the limit — leave 20–30% headroom or the container gets OOM-killed.

TIP

Set the container memory limit and let `MaxRAMPercentage` derive the heap. One place to tune, and it scales if you change the limit.

Let the JVM see the container

Modern JVMs respect container CPU and memory limits automatically. Size the heap with MaxRAMPercentage and leave headroom for non-heap memory so the container is not OOM-killed.

© 2026 ByteMart Digital

MODULE 2 · CONTAINERIZING JAVA WELL

CHAPTER 07

Building Images without a Dockerfile

Jib and Cloud Native Buildpacks — producing optimized images straight from your build tool.

Writing and maintaining Dockerfiles is not the only option. Two tools build optimized container images for Java directly from Maven or Gradle — no Dockerfile, no running Docker daemon required for Jib.

Cloud Native Buildpacks

Spring Boot has built-in support for **buildpacks**: one command produces a production-grade, layered image with a sensible base and JVM configuration — no Dockerfile at all.

bash

```
mvn spring-boot:build-image \
  -Dspring-boot.build-image.imageName=bytemart/orders:1.0
```

Jib

Jib (from Google) builds optimized, layered images from your build tool without a Docker daemon and without a Dockerfile. It separates dependencies from classes into layers automatically for fast rebuilds.

xml

```
<plugin>
  <groupId>com.google.cloud.tools</groupId>
  <artifactId>jib-maven-plugin</artifactId>
</plugin>
<!-- then: mvn jib:build -Dimage=bytemart/orders:1.0 -->
```

Which to choose

Approach	Best when
Dockerfile	you need full control
Buildpacks	you want a good image with zero config
Jib	fast, daemonless, layered builds from Maven/Gradle

Let the build tool build the image

Buildpacks (mvn spring-boot:build-image) and Jib produce optimized, layered Java images without a Dockerfile — great defaults and fast rebuilds, with Dockerfiles reserved for full control.

© 2026 ByteMart Digital

MODULE 2 · CONTAINERIZING JAVA WELL

CHAPTER 08

Image Optimization & Security

Smaller, safer images: minimal bases, non-root users, and vulnerability scanning.

A smaller image is faster to pull, cheaper to store, and has less to attack. A secure image runs with least privilege. Both are easy wins once you know the levers.

Choosing a base image

- **JRE, not JDK** — you do not need a compiler at runtime.
- **Slim or distroless** — distroless images ship only the JRE and your app, with no shell or package manager to exploit.
- **Pinned tags** — reproducible and auditable.

Run as non-root

By default containers run as root, which is dangerous if the process is compromised. Create and switch to an unprivileged user.

```dockerfile
RUN useradd -r -u 1001 appuser
USER appuser
```

WATCH OUT

Running as root inside a container is a common, avoidable risk. A compromised root process has far more power to break out or damage the host.

Scanning for vulnerabilities

Scan images for known CVEs in your CI pipeline with tools like Trivy or Grype, and rebuild on a fresh base regularly so security patches to the base OS and JRE flow through.

```bash
trivy image bytemart/orders:1.0
```

Small and least-privilege

Use a minimal JRE or distroless base, pin versions, run as a non-root user, and scan images for vulnerabilities in CI. Small, hardened images are faster and safer.

MODULE 2 · CONTAINERIZING JAVA WELL

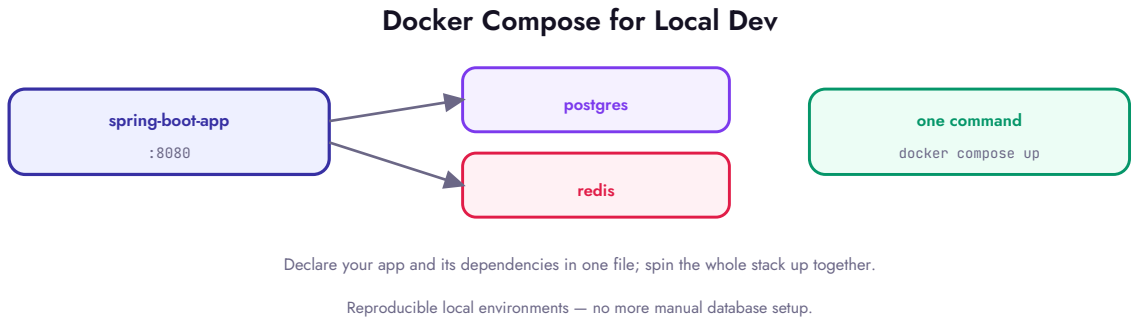
CHAPTER 09

Docker Compose for Local Dev

Spinning up your app plus its databases and dependencies with a single command.

Real apps need a database, a cache, maybe a message broker. **Docker Compose** defines your app and its dependencies in one file and runs them together — a reproducible local environment in one command.

A compose file



Declare the app and its dependencies once; bring the whole stack up together.

```yaml
services:
  app:
    build: .
    ports: ["8080:8080"]
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/orders
    depends_on: [db]
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: orders
      POSTGRES_PASSWORD: secret
    volumes: ["pgdata:/var/lib/postgresql/data"]
volumes:
  pgdata:
```

One command up and down

```bash
docker compose up -d      # start the whole stack
docker compose down       # stop and remove it
```

Why it helps teams

- New developers run one command instead of installing databases by hand.

© 2026 ByteMart Digital

- Service names (`db`) become hostnames on a shared network — no IP juggling.
- The same stack runs in CI for integration tests.

NOTE

Compose is for local development and testing, not production orchestration. For production, Kubernetes takes over — which is where the rest of this guide goes.

The whole stack in one file

Docker Compose declares your app plus its databases and dependencies and runs them with one command — reproducible local environments for development and CI, before Kubernetes takes over in production.

© 2026 ByteMart Digital

MODULE 3 · KUBERNETES FUNDAMENTALS

CHAPTER 10

Why Kubernetes

The orchestration problem that appears once you run many containers in production.

Running one container is easy. Running dozens across many machines — keeping them healthy, scaled, updated, and networked — is not. **Kubernetes** is the orchestrator that manages that for you.

What orchestration solves

- **Scheduling** — placing containers on machines with capacity.
- **Self-healing** — restarting crashed containers, replacing dead nodes.
- **Scaling** — adding and removing instances with demand.
- **Networking** — giving services stable addresses and load balancing.
- **Rollouts** — updating without downtime, rolling back on failure.

Declarative, not imperative

You do not tell Kubernetes how to run things step by step. You declare the **desired state** — 'run three replicas of this image' — and Kubernetes continuously reconciles reality to match, restarting and rescheduling as needed.

Kubernetes
An open-source container orchestrator that automates deploying, scaling, healing, and networking containerised applications across a cluster of machines, driven by declared desired state.

Is it always worth it?

WATCH OUT
Kubernetes is powerful but complex. A single small service may be better on a simpler platform. Reach for Kubernetes when you have many services, real scaling needs, and the operational maturity to run it.

Orchestration at scale
Kubernetes automates scheduling, healing, scaling, networking, and rollouts for many containers. You declare desired state and it reconciles — invaluable at scale, overkill for a single small service.

© 2026 ByteMart Digital

MODULE 3 · KUBERNETES FUNDAMENTALS

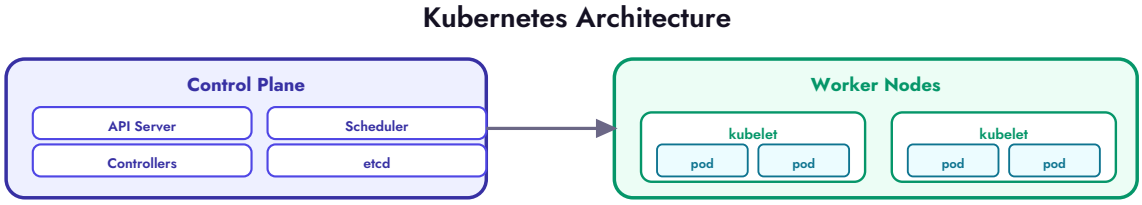
CHAPTER 11

Kubernetes Architecture

The control plane and worker nodes, and how declared state becomes running pods.

A Kubernetes cluster has two kinds of machine: the **control plane** that makes decisions, and **worker nodes** that run your containers. Understanding the split demystifies how it all works.

The two halves



The control plane decides what should run; worker nodes run the pods.

You declare desired state; Kubernetes continuously reconciles reality to match.

The control plane decides what should run; worker nodes run the pods and report back.

Control plane components

Component	Role
API Server	the front door; all requests go through it
etcd	the cluster's state database
Scheduler	assigns pods to nodes
Controllers	reconcile actual state to desired

On each worker node

- **kubelet** — the agent that runs and monitors pods on the node.
- **container runtime** — actually runs the containers (containerd).
- **kube-proxy** — handles pod networking and service routing.

The reconciliation loop

You submit desired state to the API server; it is stored in etcd. Controllers notice the gap between desired and actual and act — the scheduler places new pods, the kubelet starts them. This control loop runs continuously, which is why Kubernetes self-heals.

© 2026 ByteMart Digital

Brains and brawn

The control plane (API server, etcd, scheduler, controllers) decides and stores desired state; worker nodes (kubelet, runtime, proxy) run the pods. A continuous reconciliation loop keeps them aligned.

© 2026 ByteMart Digital

MODULE 3 · KUBERNETES FUNDAMENTALS

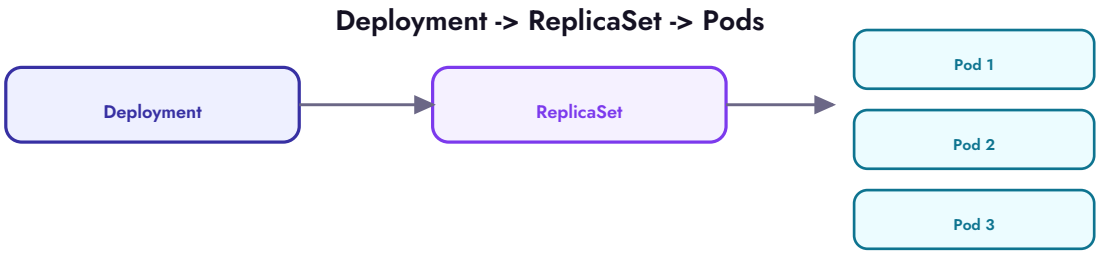
CHAPTER 12

Pods, ReplicaSets & Deployments

The objects you actually work with to run and manage your Java application.

You rarely touch containers directly in Kubernetes. Instead you work with a small hierarchy of objects — pods, ReplicaSets, and Deployments — that manage them for you.

The hierarchy



A Deployment manages a ReplicaSet, which keeps the desired number of Pods running.

Object	Purpose
Pod	smallest unit; one (or more) containers
ReplicaSet	keeps N identical pods running
Deployment	manages ReplicaSets; handles rollouts

The Pod

A **pod** is the smallest deployable unit — usually a single container (your Spring Boot app) plus its shared network and storage. Pods are ephemeral: if one dies, it is replaced by a new one with a new identity.

You deploy a Deployment

In practice you write a **Deployment**, which creates and manages a ReplicaSet to maintain your desired replica count and orchestrates updates. It is the object you apply and edit.

© 2026 ByteMart Digital

yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: { name: orders }
spec:
  replicas: 3
  selector: { matchLabels: { app: orders } }
  template:
    metadata: { labels: { app: orders } }
    spec:
      containers:
        - name: orders
          image: bytemart/orders:1.0
          ports: [{ containerPort: 8080 }]
```

Deployments manage pods for you

A Pod runs your container, a ReplicaSet keeps N pods alive, and a Deployment manages ReplicaSets and rollouts. You write Deployments; Kubernetes handles the pods beneath.

MODULE 3 · KUBERNETES FUNDAMENTALS

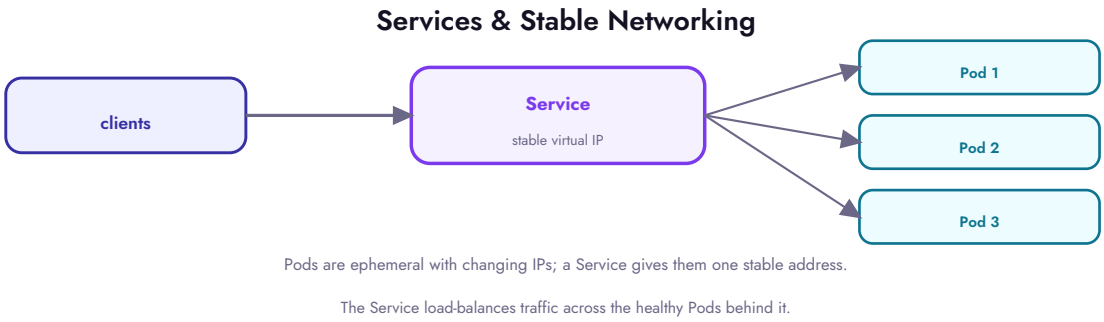
CHAPTER 13

Services & Networking

Giving ephemeral pods a stable address and load-balancing traffic across them.

Pods come and go, each with a new IP. Nothing could rely on a pod's address directly. A **Service** solves this by providing one stable endpoint in front of a changing set of pods.

The problem a Service solves



A Service gives a stable virtual IP and load-balances across the healthy pods behind it.

A Service selects pods by label and gives them a single stable virtual IP and DNS name. Traffic to the Service is load-balanced across the matching, healthy pods — callers never track individual pod IPs.

Service types

Type	Exposes the service
ClusterIP	inside the cluster only (default)
NodePort	on a port of every node
LoadBalancer	via a cloud load balancer

A Service definition

```yaml
apiVersion: v1
kind: Service
metadata: { name: orders }
spec:
  selector: { app: orders }
  ports:
    - port: 80
      targetPort: 8080
```

Service discovery

Within the cluster, a Service is reachable by its name via DNS — `http://orders` resolves to the Service. This is how one Spring Boot service calls another without hard-coded addresses.

One stable address for many pods

A Service fronts a changing set of pods with a stable IP and DNS name, load-balancing across them. ClusterIP for internal, NodePort/LoadBalancer for external — and DNS gives you service discovery.

© 2026 ByteMart Digital

MODULE 3 · KUBERNETES FUNDAMENTALS

CHAPTER 14

ConfigMaps & Secrets

Externalizing configuration and sensitive values out of your image.

The same image should run in dev, staging, and production — so configuration must live outside it. Kubernetes provides **ConfigMaps** for plain config and **Secrets** for sensitive values.

ConfigMaps

A ConfigMap holds non-sensitive configuration as key-value pairs, injected into pods as environment variables or mounted files.

yaml

```yaml
apiVersion: v1
kind: ConfigMap
metadata: { name: orders-config }
data:
  SPRING_PROFILES_ACTIVE: prod
  APP_FEATURE_X_ENABLED: "true"
```

Secrets

A Secret holds sensitive data (passwords, API keys, certificates). It is handled more carefully than a ConfigMap, though base64 encoding is not encryption — enable encryption at rest and restrict access.

yaml

```yaml
apiVersion: v1
kind: Secret
metadata: { name: orders-secrets }
type: Opaque
stringData:
  SPRING_DATASOURCE_PASSWORD: super-secret
```

Injecting into a pod

yaml

```yaml
envFrom:
  - configMapRef: { name: orders-config }
  - secretRef:    { name: orders-secrets }
```

WATCH OUT

Secrets are only base64-encoded by default, not encrypted. Enable encryption at rest, restrict RBAC access, and consider an external secrets manager for anything truly sensitive.

© 2026 ByteMart Digital

Config outside the image

ConfigMaps carry plain configuration and Secrets carry sensitive values, both injected as env vars or files. This keeps one image portable across environments — but secure your Secrets properly.

© 2026 ByteMart Digital

MODULE 4 · RUNNING JAVA ON KUBERNETES

CHAPTER 15

Deploying a Spring Boot App

Putting the pieces together to run a real Spring Boot service on Kubernetes.

15

You now have all the pieces — image, Deployment, Service, config. Let us assemble them into a working deployment of a Spring Boot application, the way you would in practice.

The full picture

- A **Deployment** runs N replicas of your image.
- A **Service** fronts them with a stable address.
- A **ConfigMap**/**Secret** supplies configuration.
- **Probes** and **resources** (next chapters) keep it healthy and bounded.

Applying manifests

You declare objects in YAML and apply them. `kubectl apply` is declarative — it creates or updates objects to match your files.

bash

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl get pods -l app=orders
kubectl logs -l app=orders --tail=50
```

Inspecting and debugging

bash

```
kubectl describe deployment orders   # events, status, rollout state
kubectl exec -it <pod> -- sh          # shell into a running pod
kubectl port-forward svc/orders 8080:80  # reach it locally
```

A repeatable workflow

Build image → push to registry → update the image tag in the Deployment → `kubectl apply`. In a real pipeline this is automated by CI/CD, but the manual flow is the same underneath.

Assemble and apply
A real deployment combines a Deployment, Service, and config, applied with kubectl apply. Use describe, logs, exec, and port-forward to inspect and debug — the same flow your CI/CD automates.

© 2026 ByteMart Digital

MODULE 4 · RUNNING JAVA ON KUBERNETES

CHAPTER 16

Health Probes for Spring Boot

Liveness, readiness, and startup probes wired to Spring Boot Actuator.

16

Kubernetes needs to know whether your app is alive, ready for traffic, and finished starting. **Probes** answer those questions — and Spring Boot Actuator provides the endpoints out of the box.

The three probes



Startup gates slow JVM boot; readiness gates traffic; liveness triggers a restart if unhealthy.

Probe	If it fails
Startup	keep waiting (do not kill a slow-starting JVM)
Readiness	stop sending traffic to the pod
Liveness	restart the container

Spring Boot support

Actuator exposes `/actuator/health/liveness` and `/actuator/health/readiness`, and enables these probe groups automatically when it detects Kubernetes. Readiness reflects whether the app can serve; liveness whether it needs a restart.

```yaml
livenessProbe:
  httpGet: { path: /actuator/health/liveness, port: 8080 }
readinessProbe:
  httpGet: { path: /actuator/health/readiness, port: 8080 }
startupProbe:
  httpGet: { path: /actuator/health/readiness, port: 8080 }
  failureThreshold: 30
  periodSeconds: 5
```

WATCH OUT

The JVM can be slow to start. Without a **startup probe**, an aggressive liveness probe may kill the app before it finishes booting, causing a restart loop. Use a startup probe to protect slow starts.

Tell Kubernetes your app's state

Startup, readiness, and liveness probes let Kubernetes manage your app correctly. Wire them to Actuator's health groups — and always add a startup probe so a slow JVM boot is not mistaken for a failure.

MODULE 4 · RUNNING JAVA ON KUBERNETES

CHAPTER 17

Resource Requests & Limits

Reserving CPU and memory sensibly — and aligning them with the JVM heap.

Every pod should declare how much CPU and memory it needs. **Requests** guarantee a minimum for scheduling; **limits** cap the maximum. Getting these right — and matching them to the JVM — is essential for stable Java pods.

Requests vs limits

Setting	Meaning
request	guaranteed minimum; used for scheduling
limit	hard maximum; exceeding memory = OOM-kill

yaml

```yaml
resources:
  requests: { cpu: "500m", memory: "512Mi" }
  limits:   { cpu: "1",    memory: "768Mi" }
```

Aligning the JVM with the limit

The JVM reads the container's memory limit, so set the limit and let `-XX:MaxRAMPercentage` derive the heap — leaving room for non-heap memory. Heap plus non-heap must fit under the limit, or Kubernetes OOM-kills the pod.

WATCH OUT

A memory limit set to exactly the heap size guarantees an OOM-kill — the JVM needs metaspace, threads, and native memory too. Budget the limit as heap plus 20–30% headroom.

CPU behaves differently

Exceeding a CPU limit does not kill the pod — it is **throttled**. Over-tight CPU limits cause latency spikes and slow garbage collection. Set generous CPU limits (or none) and rely on requests for fair scheduling.

Right-size and leave headroom

Requests reserve resources for scheduling; limits cap them. Match the memory limit to heap plus non-heap headroom via MaxRAMPercentage, and keep CPU limits generous since CPU is throttled, not killed.

© 2026 ByteMart Digital

MODULE 4 · RUNNING JAVA ON KUBERNETES

CHAPTER 18

Scaling: Manual & Autoscaling

Scaling replicas by hand and automatically with the Horizontal Pod Autoscaler.

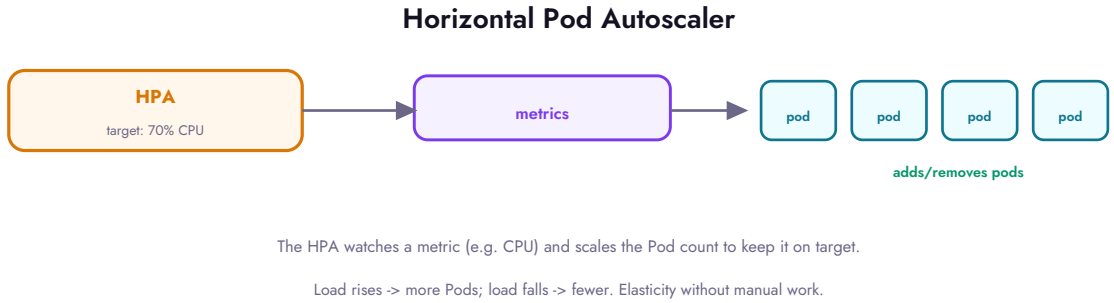
One of Kubernetes' biggest wins is effortless scaling. You can change the replica count by hand, or let the **Horizontal Pod Autoscaler** do it automatically based on load.

Manual scaling

```bash
kubectl scale deployment orders --replicas=5
```

Kubernetes schedules the new pods, the Service starts load-balancing to them, and — because your app is stateless — they share the load immediately.

The Horizontal Pod Autoscaler



The HPA watches a metric and scales the pod count to keep it near a target.

The HPA watches a metric (CPU utilisation, or a custom metric) and adjusts the replica count to keep it near a target — adding pods under load and removing them when it subsides.

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: orders }
spec:
  scaleTargetRef: { kind: Deployment, name: orders }
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 70 } }
```

TIP

Autoscaling on CPU requires accurate resource **requests** — the HPA measures usage relative to the request. Wrong requests make autoscaling misbehave.

Statelessness is the enabler

Horizontal scaling only works cleanly if your app is stateless — no in-memory session state, no local files that must persist. Externalise state to a database or cache so any pod can serve any request.

Scale by hand or automatically

Change replicas with kubectl scale, or let the HPA scale on CPU or custom metrics between min and max. Accurate resource requests and a stateless app are what make autoscaling work.

© 2026 ByteMart Digital

MODULE 4 · RUNNING JAVA ON KUBERNETES

CHAPTER 19

Configuration & Profiles

Managing Spring profiles and externalized configuration across environments.

The same image runs everywhere, so per-environment behaviour comes from configuration. Spring's profiles and externalized config map cleanly onto Kubernetes ConfigMaps and Secrets.

Profiles per environment

Activate a Spring profile through an environment variable from a ConfigMap, so `application-prod.yml` applies in production and `application-dev.yml` locally — one image, different behaviour.

yaml

```yaml
env:
  - name: SPRING_PROFILES_ACTIVE
    valueFrom:
      configMapKeyRef: { name: orders-config, key: SPRING_PROFILES_ACTIVE }
```

Relaxed binding

Spring Boot maps environment variables to properties via relaxed binding: `SPRING_DATASOURCE_URL` sets `spring.datasource.url`. This is why ConfigMap and Secret keys can drive any property without code changes.

Mounting config as files

For larger configuration, mount a ConfigMap as a file into the container and point Spring at it — useful when you prefer a full `application.yml` over many environment variables.

yaml

```yaml
volumeMounts:
  - name: config
    mountPath: /config
volumes:
  - name: config
    configMap: { name: orders-appyml }
```

TIP

Spring Cloud Kubernetes can read ConfigMaps and Secrets as property sources directly, and even reload on change — handy for larger deployments.

One image, many environments

Drive per-environment behaviour with Spring profiles activated from ConfigMaps, and set any property via relaxed binding from ConfigMaps and Secrets. Mount config as files when env vars get unwieldy.

© 2026 ByteMart Digital

MODULE 5 · PRODUCTION OPERATIONS

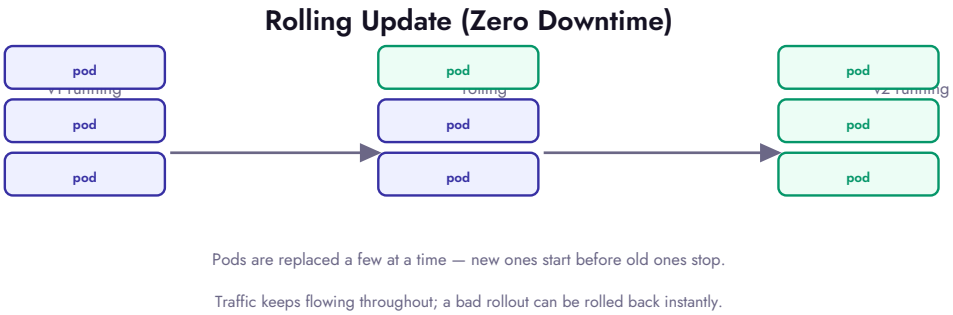
CHAPTER 20

Rolling Updates & Rollbacks

Deploying new versions without downtime — and undoing a bad release instantly.

Deploying a new version should not take your service down. Kubernetes' default **rolling update** replaces pods gradually, and a bad release can be rolled back in one command.

How a rolling update works



Pods are replaced a few at a time — new ones become ready before old ones are removed.

Kubernetes brings up new-version pods and, as each becomes **ready**, removes an old one — so capacity is maintained and traffic never hits a pod that is not ready. This is why correct readiness probes are essential to safe rollouts.

```yaml
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1          # one extra pod during rollout
    maxUnavailable: 0    # never drop below desired capacity
```

Rolling back

If a new version misbehaves, roll back to the previous revision immediately — Kubernetes keeps rollout history.

```bash
kubectl rollout status deployment/orders   # watch progress
kubectl rollout undo deployment/orders     # revert to previous
```

Graceful shutdown

During a rollout, old pods receive a termination signal. Enable Spring Boot graceful shutdown so in-flight requests finish, and set an adequate termination grace period so pods drain cleanly.

properties

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=30s
```

Update safely, undo instantly

Rolling updates replace pods gradually using readiness to protect traffic, and kubectl rollout undo reverts a bad release. Enable graceful shutdown so draining pods finish in-flight requests.

© 2026 ByteMart Digital

MODULE 5 · PRODUCTION OPERATIONS

CHAPTER 21

Ingress & Exposing Services

Routing external traffic to your services through a single, smart entry point.

A `LoadBalancer` Service per app gets expensive and unwieldy. An **Ingress** provides one entry point that routes external HTTP traffic to many services by host and path.

What an Ingress does



One entry point routes external traffic to internal Services by host and path.

TLS termination and a single load balancer instead of one per service.

One Ingress routes external traffic to internal Services by host and path, with TLS at the edge.

The Ingress resource defines routing rules; an **ingress controller** (such as NGINX) implements them. One load balancer fronts the whole cluster, terminating TLS and routing to the right Service.

An Ingress definition

yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: { name: bytemart }
spec:
  rules:
    - host: api.bytemart.com
      http:
        paths:
          - path: /orders
            pathType: Prefix
            backend:
              service: { name: orders, port: { number: 80 } }
```

Why use it

- **One entry point** instead of a load balancer per service.
- **Path and host routing** — send `/orders` and `/users` to different services.
- **TLS termination** — manage certificates in one place.

One smart front door

An Ingress routes external HTTP traffic to internal Services by host and path through a single controller, with centralised TLS — replacing a costly load balancer per service.

MODULE 5 · PRODUCTION OPERATIONS

CHAPTER 22

Observability

Logs, metrics, and traces for Spring Boot apps running on Kubernetes.

You cannot fix what you cannot see. Observability on Kubernetes rests on three pillars — logs, metrics, and traces — and Spring Boot Actuator plugs into each.

Logs

Write logs to standard output; the container platform collects them. A log aggregator (such as the EFK or Loki stack) gathers logs from all pods so you can search across the whole system — essential when a request spans several services.

Metrics

Add Micrometer's Prometheus registry and Actuator exposes `/actuator/prometheus`. Prometheus scrapes it, and you visualise the RED signals — rate, errors, duration — plus JVM metrics (heap, GC, threads) in Grafana.

```xml
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

Tracing

Distributed tracing (Micrometer Tracing with an exporter to Tempo, Jaeger, or Zipkin) stitches one request's spans across services into a single timeline — the way to find where latency or errors originate in a multi-service system.

The three pillars together

- **Metrics** tell you something is wrong (error rate up).
- **Traces** tell you where (which service, which call).
- **Logs** tell you why (the exception and context).

See logs, metrics, and traces

Ship logs to stdout for aggregation, expose Prometheus metrics via Actuator and Micrometer, and add distributed tracing. Together the three pillars let you find and diagnose problems across services.

MODULE 6 · ADVANCED & BEST PRACTICES

CHAPTER 23

Helm, Kustomize & GitOps

Packaging, templating, and automating Kubernetes deployments the professional way.

Hand-editing YAML per environment does not scale. Three practices — Helm, Kustomize, and GitOps — make Kubernetes deployments packaged, repeatable, and automated.

Helm: package manager

Helm packages a set of manifests into a reusable, templated **chart** with configurable values — install your whole app with one command and override settings per environment via a values file.

```bash
helm install orders ./orders-chart -f values-prod.yaml
```

Kustomize: overlays

Kustomize takes a template-free approach: a base set of manifests plus per-environment **overlays** that patch them. It is built into `kubectl` and favours plain YAML over templating.

```bash
kubectl apply -k overlays/prod
```

GitOps

GitOps makes Git the source of truth: your manifests live in a repository, and a controller (Argo CD or Flux) continuously syncs the cluster to match. You deploy by merging a commit — auditable, reviewable, and automatically reconciled.

Tool	Role
Helm	package and template charts
Kustomize	patch manifests with overlays
Argo CD / Flux	sync cluster to Git (GitOps)

Package, patch, and automate
Helm packages manifests into configurable charts, Kustomize patches them with overlays, and GitOps syncs the cluster to Git as the source of truth — the professional path from YAML sprawl to repeatable deploys.

© 2026 ByteMart Digital

MODULE 6 · ADVANCED & BEST PRACTICES

CHAPTER 24

Security & Best Practices

Hardening containers and clusters, and a checklist for production-ready Java workloads.

Running Java on Kubernetes securely and reliably comes down to a set of disciplines. This closing chapter gathers the most important ones into a practical checklist.

Container hardening

- Run as a **non-root** user; drop unnecessary Linux capabilities.
- Use a **minimal or distroless** base and pin versions.
- Set a **read-only root filesystem** where possible.
- Scan images for vulnerabilities in CI and rebuild regularly.

Cluster security

- Apply **RBAC** with least privilege — grant only what each workload needs.
- Restrict pod-to-pod traffic with **network policies**.
- Store secrets securely (encryption at rest, external secret managers).
- Keep the cluster and base images patched.

Reliability checklist

- Set resource **requests and limits**, aligned with the JVM heap.
- Configure **startup, readiness, and liveness** probes.
- Enable **graceful shutdown** and an adequate grace period.
- Run **multiple replicas** and keep the app stateless.
- Use **rolling updates** and be ready to roll back.

WATCH OUT

The most common production incidents trace back to a few misses: no resource limits (noisy-neighbour and OOM), no readiness probe (traffic to unready pods), and no startup probe (restart loops on slow JVM boot). Get these right first.

Discipline makes it production-ready

Harden containers (non-root, minimal base, scanning), secure the cluster (RBAC, network policies, secrets), and cover the reliability basics (resources, probes, graceful shutdown, replicas). These disciplines turn a running app into a production-ready one.

© 2026 ByteMart Digital